Intelligent Traffic Signal Coordination Using Multi-Agent Reinforcement Learning

A Multi-Agent Reinforcement Learning Approach to Adaptive Urban Traffic Management and Congestion Optimization

Technical Project Report

Contributors

M. Sai Teja Sree Veda (100523729038) · A. Maneesha (100523729039)

R. Sravya (100523729052) · Yusra Fateen (100523729062)

All results in this report were produced by the accompanying code on a custom Python traffic simulator (SUMO and CityFlow were not used). The system has only been evaluated in simulation; it has not been deployed or tested on real roads.

Abstract

Fixed-time traffic signals cannot react to changing demand. This project formulates network-wide signal control as a cooperative multi-agent reinforcement learning (MARL) problem and implements it end to end in Python. A custom queue-based simulator of a 3x3 grid of signalised intersections is controlled by nine tabular Q-learning agents, one per junction, trained for 1000 episodes on randomly drawn demand scenarios (five independent training runs per method). Two variants are studied: Independent Q-Learning (IQL) and a cooperative variant (Coop-IQL) that shares downstream-congestion information and part of the reward between neighbours. They are compared with Random, Fixed-time and Longest-queue-first controllers on 50 unseen, identical demand realisations (5 scenarios x 10 seeds). IQL reduced the average waiting time to 33.1 s versus 37.5 s for fixed-time control (11.8% lower) and 33.8 s for the longest-queue heuristic (1.9% lower), with the largest gains under asymmetric and high demand. The proposed cooperative variant did **not** improve on IQL in our experiments (37.4 s on average): it matched IQL in 2 of 5 training runs but became unstable under the time-varying peak scenario in 3 of 5 runs. We report this negative result, discuss likely causes, and list the limitations of the simulator and of the study.

1. Introduction

Road congestion increases travel times, fuel use and emissions. A large part of urban delay arises at signalised intersections, where the timing plan decides who waits. Most deployed signals use fixed-time plans or simple vehicle-actuated rules. Fixed-time plans are tuned to an average day and cannot adapt to incidents, events or rush-hour imbalance. Reinforcement learning (RL) offers a way to learn a control policy directly from interaction with traffic, using only a reward signal that expresses what 'good traffic' means.

Because a city has many intersections, each signal can be treated as an agent. Neighbouring agents influence each other: a long green at one junction releases a platoon that arrives at the next, and a full downstream link blocks the upstream junction. This makes traffic-signal coordination a canonical multi-agent RL problem.

2. Problem Definition

Given a network of interconnected signalised intersections with stochastic vehicle arrivals, learn a decentralised signal-control policy - one agent per intersection - that reduces average vehicle waiting time, queue length and congestion, and increases throughput, compared with fixed-time control, using only locally observable information and limited communication with neighbouring agents.

3. Objectives

(1) Build a reproducible multi-intersection simulator with variable demand. (2) Implement independent RL agents with state, action and reward definitions suited to traffic. (3) Add cooperation between neighbouring agents through information and reward sharing. (4) Compare against Random, Fixed-time and Longest-queue-first baselines on identical traffic. (5) Measure waiting time, queue length, throughput, travel time and congestion, and report the results - including unfavourable ones - honestly.

4. Related Technical Concepts

Classical signal control. Webster (1958) derived fixed-time cycle lengths and splits that minimise delay for a given average flow. Actuated and adaptive systems extend this idea with detectors. Max-pressure control (Varaiya, 2013) is a decentralised rule with stability guarantees that serves the phase with the largest pressure (upstream minus downstream queue). Our Longest-queue-first baseline is a simpler actuated heuristic in the same spirit.

RL for signals. Early work applied Q-learning to a single junction (Abdulhai et al., 2003) and to several junctions with cooperating agents (Wiering, 2000). Tan (1993) discussed independent versus cooperative

learners, the basis of the IQL and cooperative variants here. More recent work uses deep networks, for example PressLight (Wei et al., 2019) and multi-agent actor-critic for large networks (Chu et al., 2019); Wei et al. (2019) also provide a survey. SUMO (Lopez et al., 2018) is the usual open-source simulator in this literature. We use tabular Q-learning, which is simple and transparent but less powerful than those deep methods.

Reinforcement learning fundamentals

An RL problem is a Markov decision process (MDP) (S, A, P, R, γ): at each step the agent observes state s, chooses action a, receives reward r and moves to s' with probability P(s'|s,a). The goal is a policy $\pi(a|s)$ maximising the expected discounted return $G = \Sigma\, \gamma^t\, r_t$ (Sutton & Barto, 2018). The action-value function Q(s,a) is the expected return of taking a in s and following π afterwards.

Q-learning (Watkins & Dayan, 1992) is an off-policy temporal-difference method: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. Exploration uses ε-greedy action selection. In a **multi-agent** setting, each agent sees an environment that changes as the other agents learn (non-stationarity), so the single-agent convergence guarantee of Q-learning no longer applies. In **Independent Q-Learning** every agent simply ignores this and learns as if alone; it often works in practice but can be unstable.

5. System Architecture

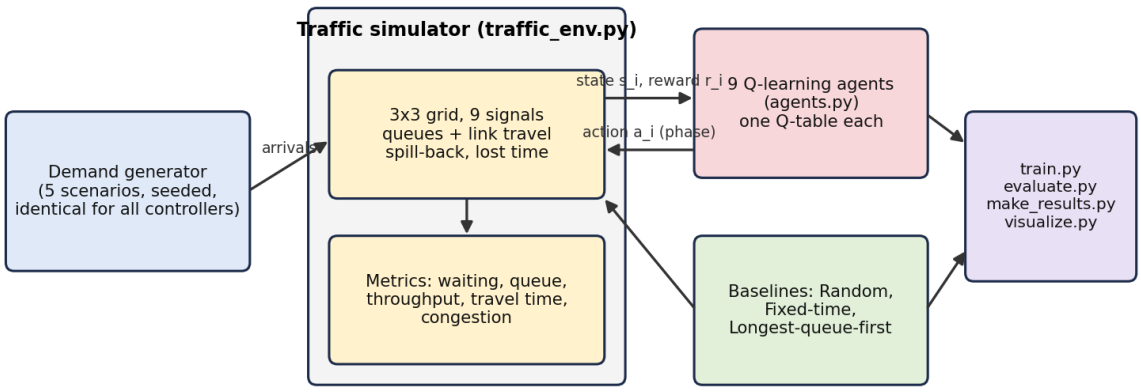


Figure 1. System architecture and files.

The demand generator pre-computes every vehicle arrival of an episode from a seed, so all controllers face exactly the same vehicles. The simulator advances in 1 s steps; agents are queried every 10 s.

6. Reinforcement Learning Methodology

6.1 MDP formulation

Each intersection i is an agent in a decentralised, partially observable multi-agent MDP (a Markov game): its observation covers only its own approaches, plus (for Coop-IQL) two flags about downstream approaches. Table 1 summarises the formulation; Figure 2 shows how neighbouring agents interact.

State	Own queue on the N+S axis and on the E+W axis, each binned to 4 levels {0-3, 4-9, 10-19, 20+}; current phase (2); longest head-of-queue waiting time on the red axis binned to 3 levels {<15 s, 15-40 s, >40 s}. = 96 states (IQL). Coop-IQL appends two binary flags (per axis: is any downstream approach it feeds ≥60% full, i.e. ≥12 of 20 vehicles?) = 384 states.
Action	Phase for the next 10 s: 0 = north-south green, 1 = east-west green. Changing phase inserts 2 s of all-red (lost time).
Reward	$r_i = -(0.10\, q_{avg,i} + 0.50\, w_{max,i}/60 + 0.50 \min(b_i/5, 2))$. $q_{avg,i}$: mean queued vehicles at junction i during the 10 s; $w_{max,i}$: longest head-of-queue wait; b_i: blocked discharge attempts (spill-back).

Shared reward (Coop-IQL)	$\tilde{r}_i = 0.7\, r_i + 0.3 \cdot \text{mean}_{j \in N(i)}\, r_j$, N(i) = adjacent junctions.
Transitions	Stochastic (random arrivals, other agents' behaviour). Time-limit episodes of 180 decisions; the update bootstraps at the last step.
Discount	$\gamma = 0.9$ (about a 100 s horizon).
Objective	Maximise $E[\Sigma\, \gamma^t\, r_{i,t}]$ for every i; network goal: minimise mean vehicle waiting time.

Table 1. MDP formulation for each agent.

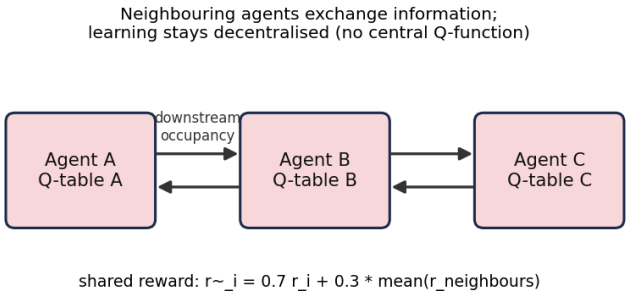


Figure 2. Neighbouring agents share information and part of their reward; each keeps its own Q-table.

6.2 Algorithm and training procedure

Algorithm. Nine tabular Q-learners, one per intersection, with separate Q-tables (intersections on the border and at the centre face different traffic, so sharing one table was not assumed). All agents act simultaneously every 10 s. Hyper-parameters: $\alpha = 0.1$, $\gamma = 0.9$, ε decayed linearly from 1.0 to 0.05 over the first 700 of 1000 episodes, co-operation weight $\beta = 0.3$. Training episodes use a randomly drawn scenario (low, medium, high, asym, peak) and a random demand seed from 0-999, so the policy must generalise across demand patterns. Test demand uses seeds 1000-1009, never seen in training; validation seeds 5000-5001 are used to monitor training.

Cooperation. Coop-IQL differs from IQL in exactly two ways: the state includes the downstream-fullness flags (information sharing) and the reward blends in the neighbours' rewards (reward sharing). It is a decentralised design: no central Q-function or central controller is used.

Baselines. *Random*: a uniformly random phase every 10 s. *Fixed-time*: all junctions alternate 20 s NS / 20 s EW (40 s cycle, no offsets). *Longest-queue-first*: switch to the axis with the larger queue when it exceeds the current axis by more than 2 vehicles (extra, non-learning reference).

7. Implementation

The code is plain Python 3 with NumPy and Matplotlib (no paid APIs). Modules: `config.py` (all constants), `traffic_env.py` (simulator), `agents.py` (baselines and Q-learners), `train.py`, `evaluate.py`, `make_results.py` (charts and tables), `visualize.py` (animation), `tests/test_env.py` (5 unit tests, e.g. vehicle conservation and queue-capacity checks), `run_all.py` (one-command pipeline).

Simulator model. Each approach (vehicle moving S, N, E or W) has a FIFO queue; a green approach releases one vehicle every 2 s (saturation flow 0.5 veh/s). The vehicle then spends 8 s on the next link. A vehicle may enter an internal approach only if queue + vehicles on the link < 20, otherwise it waits (spill-back). Routes are dimension-ordered (columns, then rows) to a random boundary exit; vehicles do not re-route. Left turns, pedestrians, yellow phases and vehicle dynamics are not modelled. Demand per entry point (veh/s): low 0.07, medium 0.10, high 0.13; *asym* 0.16 for southbound/eastbound and 0.05 for the other entries; *peak* rising linearly 0.05 → 0.18 at mid-episode and back to 0.05 (about 1500-2800 vehicles per 30

min episode). The demand levels were chosen so that fixed-time control is comfortable under low demand and strained under peak demand.

8. Experimental Setup

Training	1000 episodes x 5 independent runs (seeds 0-4) for each of IQL and Coop-IQL; random scenario per episode.
Evaluation	50 demand realisations (5 scenarios x 10 seeds, 1000-1009), 1800 s each; the same vehicle arrivals for every controller. RL controllers are greedy ($\varepsilon = 0$); each of the 5 trained runs is evaluated on all 50 realisations (250 episodes per RL method), baselines 50 episodes.
Metrics	**Average waiting time**: mean over all vehicles of time spent in a queue (including vehicles still queued at the end). **Average queue length**: time-averaged vehicles per approach. **Throughput**: vehicles that left the network in 1800 s. **Average travel time**: mean over completed trips. **Congestion index**: time-averaged share of approaches holding ≥10 queued vehicles. **Spill-back events**: blocked discharge attempts.
Hardware	Single CPU core. Training of all 10 runs took about 10 minutes; evaluation about a minute.

9. Results and Discussion

Learning curves. Figure 3 (left) shows the episode return during training: both RL variants improve from about −300 to about −100 and then plateau, with nearly identical curves. The training return is noisy because every episode uses a different random scenario. Figure 3 (right) shows the greedy validation waiting time: a policy that is already competitive (≈33-35 s) appears after only about 50 episodes. Late in training the validation curves oscillate; with a constant learning rate and non-stationary neighbours, individual runs sometimes drift to worse policies, and the 'best' checkpoint is not the last. We report the final policy of every run (no cherry-picking of checkpoints).

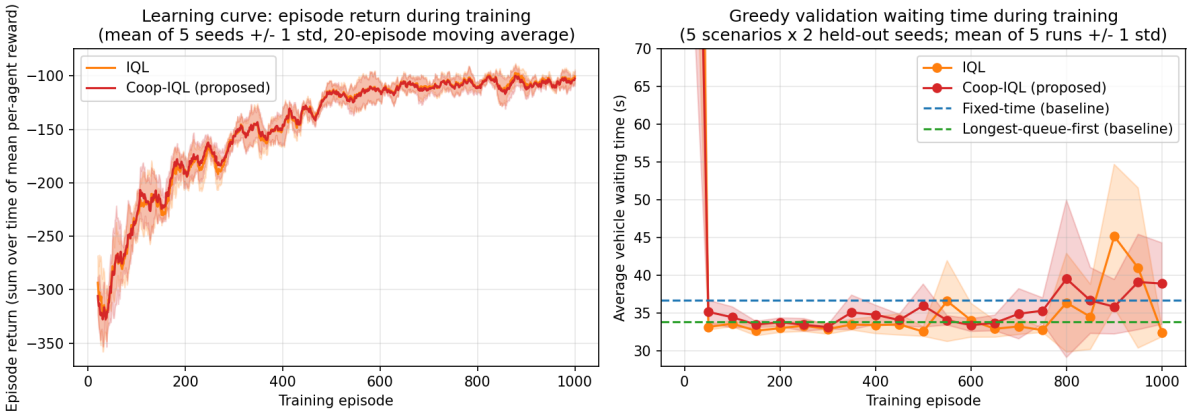


Figure 3. Left: training return (mean +/- std over 5 runs, 20-episode moving average). Right: greedy validation waiting time during training versus baselines.

Overall comparison. Table 2 and Figure 4 average over all scenarios and test runs.

Controller	Avg wait (s)	Avg queue (veh/approach)	Throughput (veh)	Avg travel time (s)	Congestion index	Spill-back events
Random	90.8	3.32	2083	106.3	0.109	410
Fixed-time	37.5	1.35	2179	55.8	0.019	30
Longest-queue-first	33.8	1.19	2190	52.2	0.005	4
IQL	33.1	1.18	2188	50.9	0.007	23
Coop-IQL (proposed)	37.4	1.35	2169	52.3	0.012	75

Table 2. Mean results over 5 scenarios x 10 test seeds (RL rows also averaged over 5 training runs).

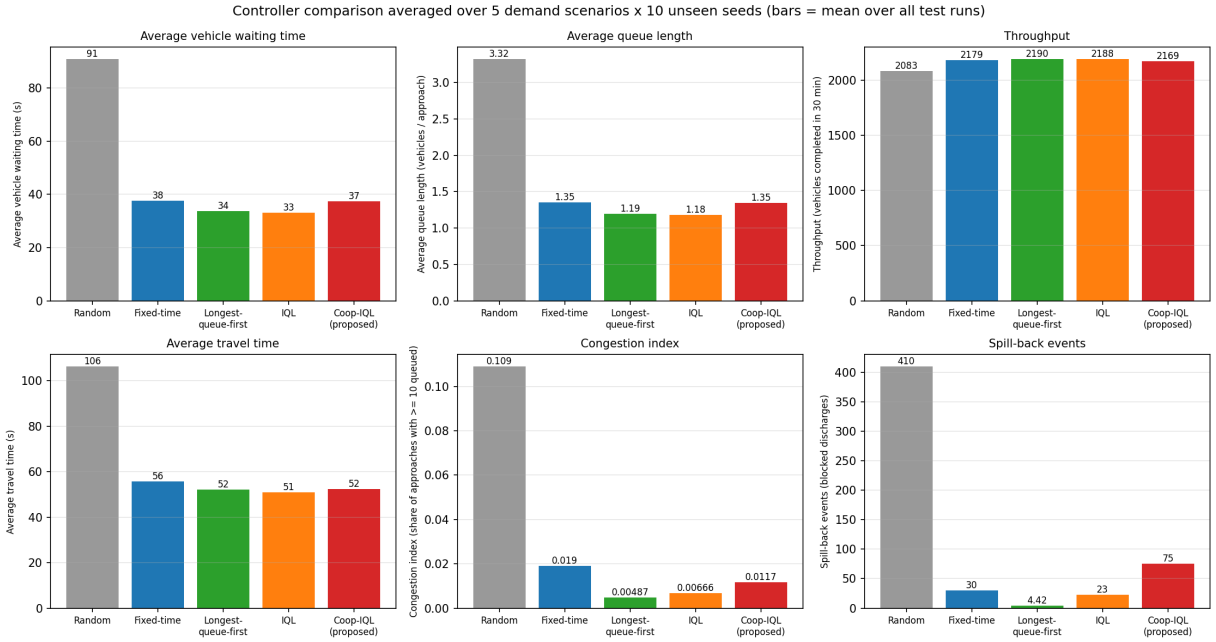


Figure 4. Overall comparison of all controllers on the five metrics (plus spill-back events).

IQL achieved the lowest average waiting time (33.1 s), 11.8% below Fixed-time and 1.9% below Longest-queue-first, and 63.5% below Random. Its average queue length (1.18 vehicles per approach) and travel time (50.9 s) were also the lowest. Throughput differs little between sensible controllers (about 2179-2190 vehicles), because in 30 minutes almost all vehicles eventually get through when the network is not gridlocked; the benefit of good control appears mainly in waiting time and queues, not in total vehicles served. Random control was clearly the worst.

By scenario. Table 3 and Figures 5-7 show that the advantage depends on demand.

Controller	low	medium	high	asym	peak
Random	44.5	60.0	108.6	112.2	129.0
Fixed-time	23.1	27.1	38.4	47.1	52.0
Longest-queue-first	27.9	28.6	33.6	33.8	45.0
IQL	23.5	26.4	33.5	33.9	48.4
Coop-IQL (proposed)	24.0	26.9	34.3	34.5	67.5

Table 3. Average waiting time (s) by demand scenario.

Controller	low	medium	high	asym	peak
Random	1465	2050	2545	2030	2327
Fixed-time	1485	2089	2715	2168	2436
Longest-queue-first	1485	2095	2728	2206	2435
IQL	1490	2097	2730	2205	2419
Coop-IQL (proposed)	1489	2096	2729	2204	2329

Table 4. Throughput (vehicles in 30 min) by demand scenario.

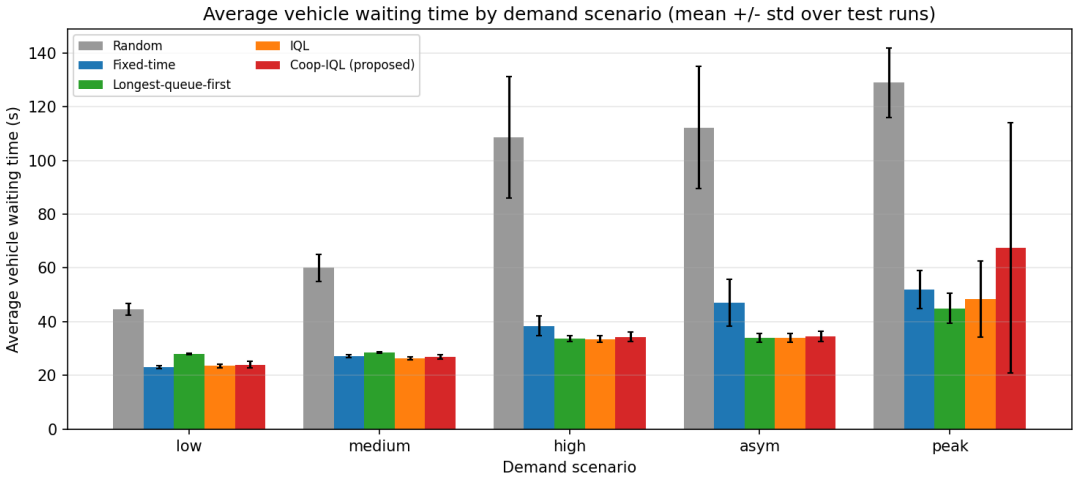


Figure 5. Average waiting time by demand scenario.

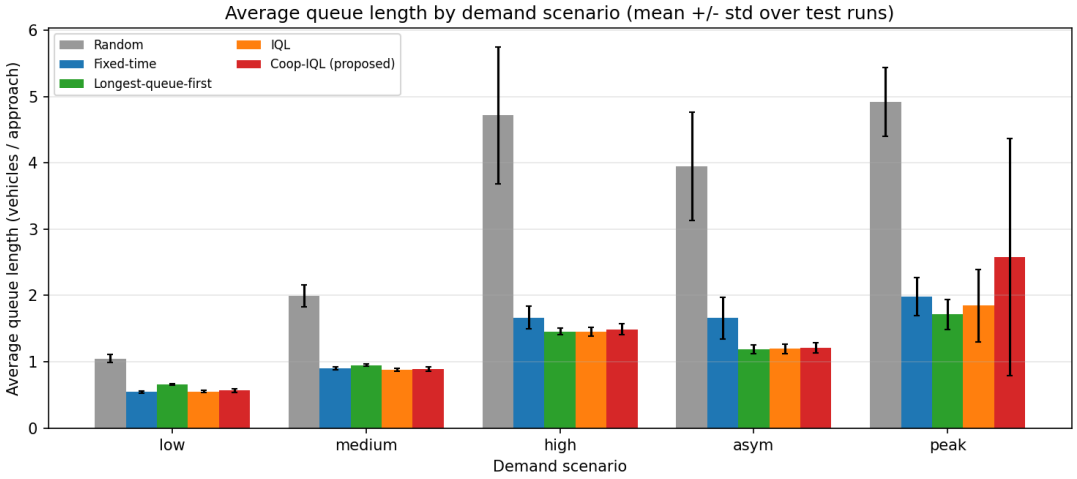


Figure 6. Average queue length by demand scenario.

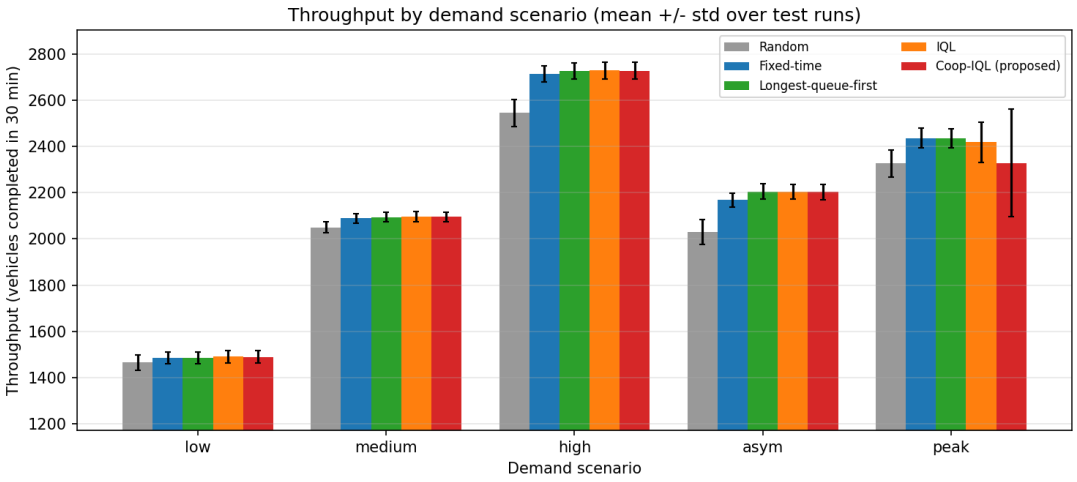


Figure 7. Throughput by demand scenario (the y-axis starts at 80% of the smallest value).

Under low and medium demand, Fixed-time and IQL are almost equivalent (23.1 vs 23.5 s at low demand), and Longest-queue-first is actually worse than Fixed-time there because its switching causes extra lost time. Learning pays off when demand is high or unbalanced: under *asym* demand IQL reached 33.9 s versus 47.1 s for Fixed-time (28% lower), because the learned policy gives more green to the heavy direction. Under *peak* demand IQL reduced waiting from 52.0 to 48.4 s.

Cooperative variant - an unexpected result. Coop-IQL averaged 37.4 s, i.e. no better than Fixed-time (paired difference -0.1 s, 95% bootstrap CI [-3.4, 3.7]) and worse than IQL (+4.3 s, CI [1.7, 7.4]). The difference is not uniform: in the low, medium, high and asym scenarios Coop-IQL is within about 1 s of IQL, but under *peak* demand 3 of 5 Coop-IQL training runs produced policies with waiting times of about 80 s, while 0 of 5 IQL runs did (per-run peak waiting times: Coop-IQL 48, 80, 48, 80, 82 s; IQL 47, 46, 55, 46, 47 s). So the average hides a bimodal outcome: some runs work as well as IQL and others do not. We did not find evidence that the cooperation mechanisms help. Plausible but *untested* explanations: (i) the larger state space (384 vs 96 states) is visited less often, especially the rarely seen 'downstream full' states that matter only during peaks; (ii) reward sharing makes each agent's reward noisier and couples learning across agents, adding to non-stationarity; (iii) the downstream flag is a coarse 0/1 signal. We did not run ablations that separate (a) from (b), so we cannot say which one hurts.

Design iteration (full disclosure). A first version of Coop-IQL, using the mean total queue of neighbouring junctions as shared information and 400 training episodes, was evaluated first (avg. waiting time: IQL 36.2 s, Coop-IQL 43.4 s; archived in `05_Experimental_Results/iteration1_v1_400ep`). Because it also failed in the peak scenario, we changed the shared information to downstream-fullness flags and trained for 1000 episodes (both methods), using validation seeds to check the change. Note that this redesign was motivated by seeing the first test-set results, so the test seeds are not completely untouched; the improvement for IQL from 400 to 1000 episodes is part of the same change. The final design is still outperformed by plain IQL.

10. Limitations

(1) **Simulator realism:** a custom queue-based model, not SUMO; no vehicle dynamics, lane changing, left turns, pedestrians, yellow phases, or re-routing, and a single network size. Absolute numbers (e.g. seconds of waiting) are properties of this simulator and must not be read as real-world predictions. (2) **Algorithm:** tabular Q-learning with hand-chosen bins; no deep RL; constant learning rate; hyper-parameters and reward weights were not tuned systematically. (3) **Baselines:** the Fixed-time plan was not optimised (no Webster calculation, no offsets/green wave), so a tuned fixed plan may be stronger than the one used here. Max-pressure was not implemented. (4) **Statistics:** 5 training runs and 50 test episodes per scenario set; the bimodal Coop-IQL outcome shows that 5 runs are few. Throughput differences between sensible controllers are small relative to scenario variance. (5) **Evaluation protocol:** test seeds were used once before redesign (Section 9). (6) The 'travel time' averages only vehicles that completed their trip, which flatters controllers that leave many vehicles in the network; the counts of vehicles left are stored in the raw results.

11. Safety and Ethical Considerations

Safety and reliability. A learned controller must never compromise safety: real deployments need hard-coded safety layers (minimum green, clearance and conflict monitors, fail-safe flashing or fixed-time fallback when sensors or software fail). Our agents have no such layer; a learned policy can fail abruptly in situations not seen in training, as the peak-scenario failures of some runs illustrate. **Fairness.** Minimising average waiting time can starve minor roads, pedestrians, cyclists and buses; we include a longest-wait term in the reward, but a real system should include these road users explicitly and audit who waits longest. **Emergency vehicles** need priority mechanisms. **Privacy.** Queue counts from loop detectors are less sensitive than camera or licence-plate data; any camera-based sensing must follow data-protection law. **Accountability and transparency.** Operators need explainable behaviour, logs, and a clear responsibility when a learned policy contributes to an accident or to inequitable delays. **Claims.** This work is a simulation study; it makes no claim about performance on real roads.

12. Future Scope

Recommended next steps: SUMO or CityFlow validation, Webster-tuned and max-pressure baselines, function approximation (DQN/actor-critic) with parameter sharing, ablations of information versus reward sharing, more training seeds, and safety layers.

13. Conclusion

We implemented and evaluated multi-agent Q-learning for coordinated traffic-signal control in a 3x3 simulated network. Independent Q-learning learned within a few hundred episodes a policy that lowered waiting time by 12% compared with a 20-s fixed-time plan overall and by about 28% under asymmetric demand, but offered little benefit at low demand and was roughly on par with a simple longest-queue heuristic (2% better overall). The cooperative variant we proposed did not improve on IQL and was less reliable under peak demand. The main lessons are that (i) even simple tabular MARL can beat fixed-time control when demand is uneven, (ii) cooperation is not automatically beneficial and needs careful design and ablation, and (iii) reliability across training seeds matters as much as average performance.

References

Abdulhai, B., Pringle, R., & Karakoulas, G. J. (2003). Reinforcement learning for true adaptive traffic signal control. *Journal of Transportation Engineering*, 129(3), 278-285.

Chu, T., Wang, J., Codeca, L., & Li, Z. (2019). Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 21(3), 1086-1095.

Lopez, P. A., et al. (2018). Microscopic traffic simulation using SUMO. *Proc. IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2575-2582.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Tan, M. (1993). Multi-agent reinforcement learning: Independent vs. cooperative agents. *Proc. 10th International Conference on Machine Learning*, 330-337.

Varaiya, P. (2013). Max pressure control of a network of signalized intersections. *Transportation Research Part C*, 36, 177-195.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279-292.

Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Her Majesty's Stationery Office, London.

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019). A survey on traffic signal control methods. *arXiv:1904.08117*.

Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019). PressLight: Learning max pressure control to coordinate traffic signals in arterial network. *Proc. ACM SIGKDD*, 1290-1298.

Wiering, M. (2000). Multi-agent reinforcement learning for traffic light control. *Proc. 17th International Conference on Machine Learning*, 1151-1158.